

Design Pattern Adapter

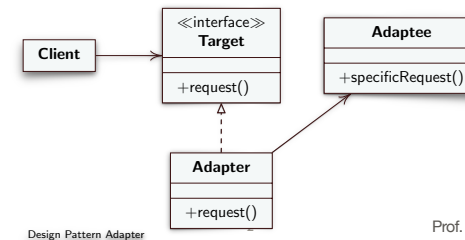
- **Intento:** Convertire l'interfaccia di una classe in un'altra interfaccia che i client si aspettano. Adapter permette ad alcune classi di interagire, eliminando il problema di interfacce incompatibili
- Problema
 - Alcune volte una classe di una libreria non può essere usata poiché incompatibile con l'interfaccia che si aspetta l'applicazione. Ovvero nome metodo, parametri, tipo parametri, di chiamate all'interno dell'applicazione non sono corrispondenti a quelli offerti da una classe di libreria
 - Non è possibile cambiare l'interfaccia della libreria, poiché non si ha il sorgente (comunque non conviene cambiarla)
 - Non è possibile cambiare l'applicazione (il chiamante), e si può voler cambiare quale metodo chiamare, senza renderlo noto al chiamante

1

Prof. Tramontana - 2 Aprile 2025

Design Pattern Adapter

- Soluzione nella versione Object Adapter
 - **Target** è l'interfaccia che il chiamante si aspetta
 - **Client** usa oggetti che sono conformi all'interfaccia Target
 - **Adaptee** è l'oggetto di libreria che necessita l'adattamento
 - **Adapter** converte, ovvero adatta, la chiamata che fa una classe client all'interfaccia della classe di libreria. Il chiamante (Client) usa l'Adapter come se fosse l'oggetto di libreria. La classe con ruolo **Adapter** implementa **Target**, tiene il riferimento all'oggetto di libreria (Adaptee) e sa come invocarlo, ovvero chiama i metodi di Adaptee

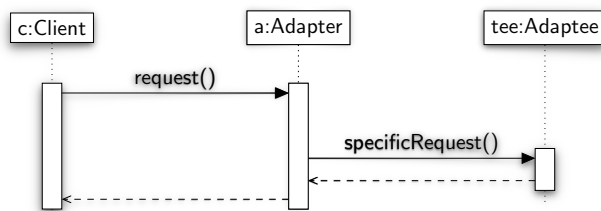


Design Pattern Adapter

Prof. Tramontana - 2 Aprile 2025

Design Pattern Adapter

- Diagramma di sequenza della soluzione Object Adapter

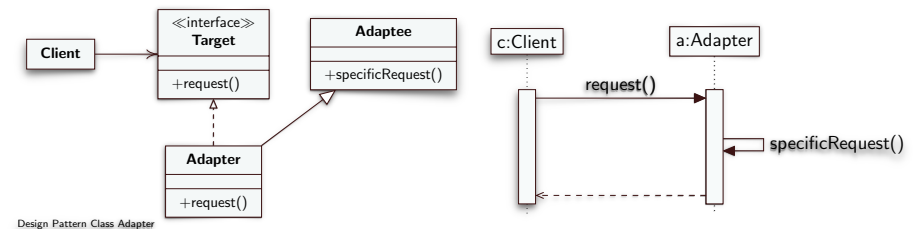


3

Prof. Tramontana - 2 Aprile 2025

Design Pattern Adapter

- Soluzione nella versione Class Adapter
 - Client, Target e Adaptee rimangono invariati rispetto alla soluzione Object Adapter
 - La classe Adapter è sottoclasse di Adaptee, implementa l'interfaccia Target e chiama i metodi della superclasse



Design Pattern Class Adapter

4

Prof. Tramontana - 2 Aprile 2025

Design Pattern Adapter

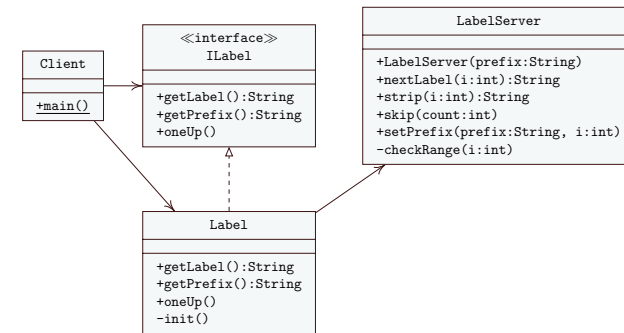
- Variante design pattern Adapter a due vie
 - Definizione: la classe con ruolo Adapter fornisce l'interfaccia di Target e l'interfaccia di Adaptee
 - Realizzazione: la soluzione Class Adapter è un Adapter a due vie
- Conseguenze del design pattern Adapter
 - Client e classe di libreria Adaptee rimangono indipendenti. Il ruolo Adapter può cambiare il comportamento dell'Adaptee
 - Può aggiungere test di **precondizioni** e **postcondizioni** [Precondizioni: cosa si deve soddisfare prima di eseguire. Postcondizioni: cosa è verificato se tutto è andato bene]
 - L'Object Adapter può implementare la tecnica di **Lazy Initialisation** (si aspetta che avvenga un'invocazione a un metodo di Adaptee prima di istanziarlo)
 - Il design pattern Adapter aggiunge un livello di indirettezza. Ogni invocazione del Client ne scatena un'altra fatta dal ruolo Adapter. Possibile rallentamento (trascurabile), e codice più difficile da comprendere.

5

Prof. Tramontana - 2 Aprile 2025

Esempio Adapter

- La classe Label (Object Adapter) adatta le chiamate dei metodi verso la classe LabelServer (Adaptee)
- La chiamata a getLabel diventa la chiamata a nextLabel con il parametro intero, la chiamata a getPrefix diventa la chiamata a strip, e la chiamata a oneUp diventa la chiamata a skip



6

Prof. Tramontana - 2 Aprile 2025

```

// Adaptee
public class LabelServer {
    private int labelNum = 1;
    private String[] labelPrefix = new String[3];

    public LabelServer(String prefix) {
        labelPrefix[0] = prefix;
        labelPrefix[1] = "";
        labelPrefix[2] = "";
    }

    public String nextLabel(int i) {
        System.out.println("In nextLabel");
        if (checkRange(i))
            return labelPrefix[i] + labelNum++;
        return null;
    }

    public String strip(int i) {
        System.out.println("LabelServer: in strip");
        if (checkRange(i)) return labelPrefix[i];
        return null;
    }

    public void skip(int count) {
        labelNum += count;
    }

    public void setPrefix(String prefix, int i) {
        System.out.println("In setPrefix");
        if (checkRange(i))
            labelPrefix[i] = prefix;
    }

    private boolean checkRange(int i) {
        return i >= 0 && i < labelPrefix.length;
    }
}
  
```

```

// Target interface
public interface ILabel {
    public String getLabel();
    public String getPrefix();
    public void oneUp();
}

// Adapter (Object version)
public class Label implements ILabel {
    private LabelServer ls;
    private final String prefix = "MyLabel";

    @Override
    public String getLabel() {
        System.out.println("Label in getLabel");
        init();
        return ls.nextLabel(0);
    }

    @Override
    public String getPrefix() {
        System.out.println("Label in getPrefix");
        init();
        return ls.strip(0);
    }

    @Override
    public void oneUp() {
        System.out.println("Label in oneUp");
        init();
        ls.skip(1);
    }

    private void init() {
        if (null == ls) ls = new LabelServer(prefix);
    }
}
  
```

7

Prof. Tramontana - 2 Aprile 2025

```

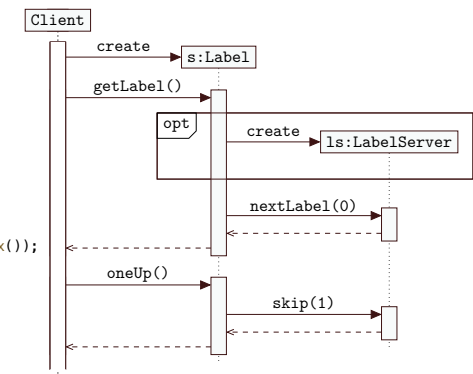
public class Client {
    public static void main(String args[]) {
        ILabel s = new Label();
        System.out.println("Istanziata Label");

        String l = s.getLabel();
        if (!l.equals("MyLabel1"))
            System.out.println("Test 1: Passed");
        else
            System.out.println("Test 1: Failed");

        System.out.println("Prefisso " + s.getPrefix());

        s.oneUp();

        l = s.getLabel();
        if (!l.equals("MyLabel3"))
            System.out.println("Test 2: Passed");
        else
            System.out.println("Test 2: Failed");
    }
}
  
```



8

Prof. Tramontana - 2 Aprile 2025

Design Pattern Adapter

- Soluzione Class Adapter
 - La classe LabelC (con il ruolo Adapter) è sottoclasse di Adaptee

```
// Adapter (Class version)
public class LabelC extends LabelServer implements ILabel {
    public LabelC() {
        super("MyLabel");
    }

    @Override
    public String getLabel() {
        System.out.println("in Adapter chiamo Adaptee");
        return nextLabel(0);
    }

    @Override
    public String getPrefix() {
        System.out.println("in Adapter chiamo Adaptee");
        return strip(0);
    }

    @Override
    public void oneUp() {
        System.out.println("in Adapter chiamo Adaptee");
        skip(1);
    }
}
```